



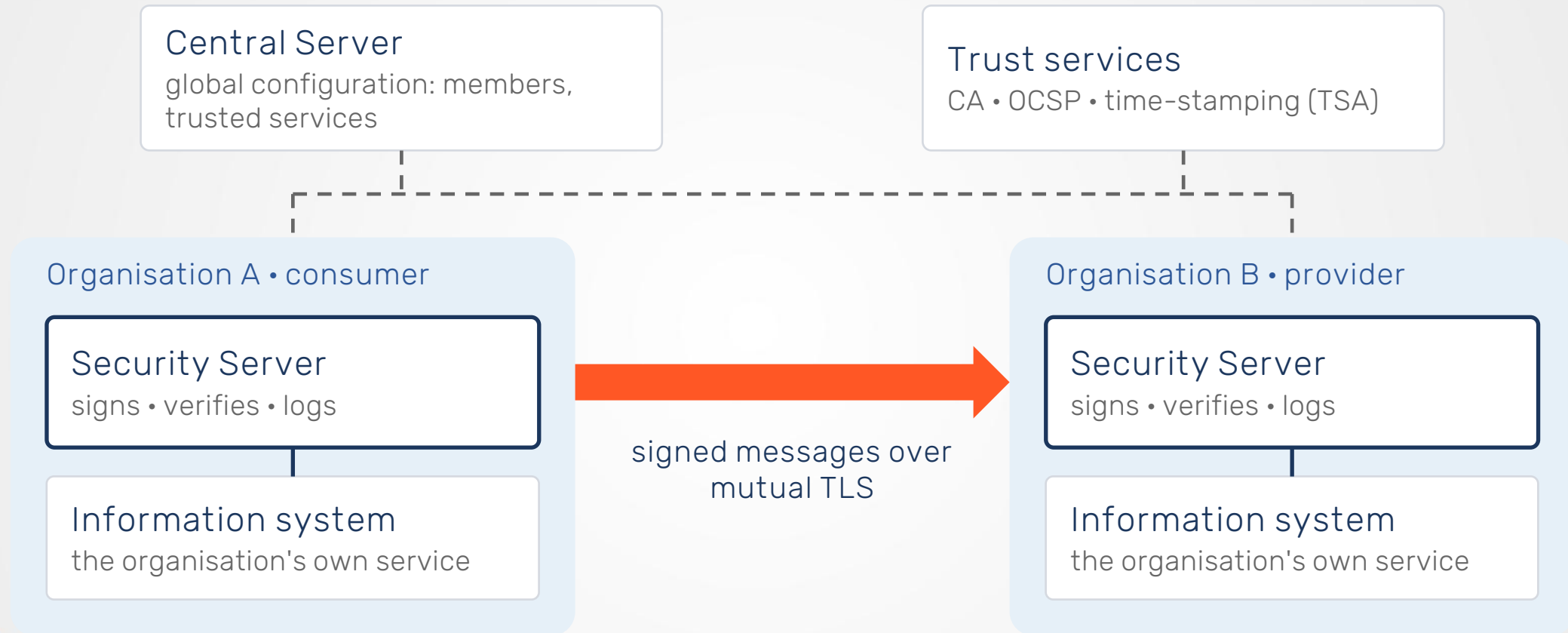
Measuring the Overhead of Trust Mechanisms in the X-Road Data Exchange Layer

Research plan, related work and preliminary measurements

Ruslan Nechshadimov • Supervisor: Artjom Lind • October 2026



What X-Road is



Dashed: used in the background, not on every message.

Research question

What does each trust mechanism of X-Road cost per message, in latency and in bytes, and which of that cost is paid on every request rather than deferred?

RQ1

Contribution of TLS, signing, verification, logging and time-stamping to latency and bytes

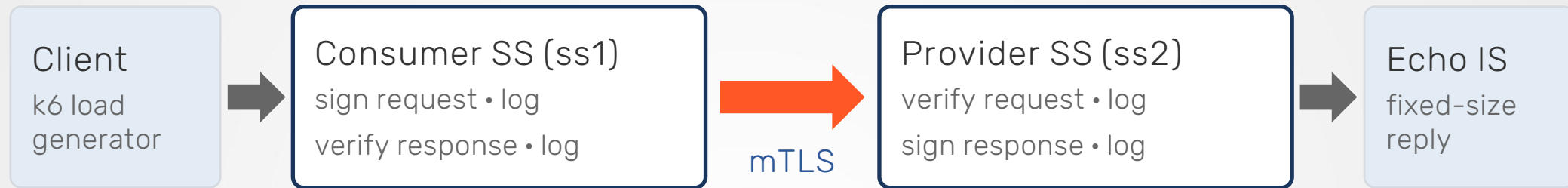
RQ2

How the contributions change with message size and with concurrency

RQ3

How much the answers depend on the environment: one host vs separate wired machines

The measured request path



Paid on every request

- signing and verification, both directions
- message log write on both servers
- TLS handshake: by default once per message, connections are not reused

Deferred or amortised

- time-stamping: batched over a hash chain, about 2 TSA calls per 1000 messages
- OCSP: cached responses, refreshed every 20 minutes
- global configuration refresh



What the closest X-Road studies measured

Work	Measured or established	Left open
Ansper et al. 2013	Design of signing, OCSP caching, batched time-stamping; analytical throughput estimate	No measured per-message cost
Mironova 2017	Central services testing; production intervals; behaviour without TSA	Cost on the message path
Taklai 2020	Service latency through the Security Server vs direct	Which mechanism the overhead comes from
Snetkov 2023	Post-quantum signatures in UXP: signature sizes	Effect on latency and throughput
Bakhtina 2026 (IJIS)	Threshold-signature round trip, one laptop	Breakdown; separate machines

Gap: measured end to end or on one machine, not per mechanism; NIIS publishes no open performance data.



Broader work used for the method

Naylor et al., CoNEXT 2014

The Cost of the “S” in HTTPS

TLS cost has two parts: connection setup and per-request work. How connections are used decides the cost per request.

Used for: separating the handshake from per-message work (configuration reuse).
HTTPS, not X-Road: numbers not compared directly.

Georges et al., OOPSLA 2007

Statistically Rigorous Java Performance Evaluation

JVM timings need warm-up, independent runs, and the spread between runs reported next to the result.

Used for: burn-in and warm-up, 5 independent repetitions per grid, randomised order, IQR and range reported.

Configurations actually run

Name	What changes	What the difference to full shows
direct	Client calls the echo service, no X-Road	Whole X-Road path (not single operations)
full	Defaults: signing, logging, batched time-stamps	Reference point
nobody	Message bodies not written to the log	Cost of logging bodies only
synctsa	Time-stamp on every message, synchronously	A different policy, not “the cost of TSA”
reuse	Connections between SS kept and reused	Per-message TLS handshake
full+ss3	Load split over two provider SS	Consumer-side limits under load

Dropped: notsa (interval outside the documented constraint) and short OCSP freshness (every request failed, gives no OCSP estimate).

Metrics and procedure

Metrics

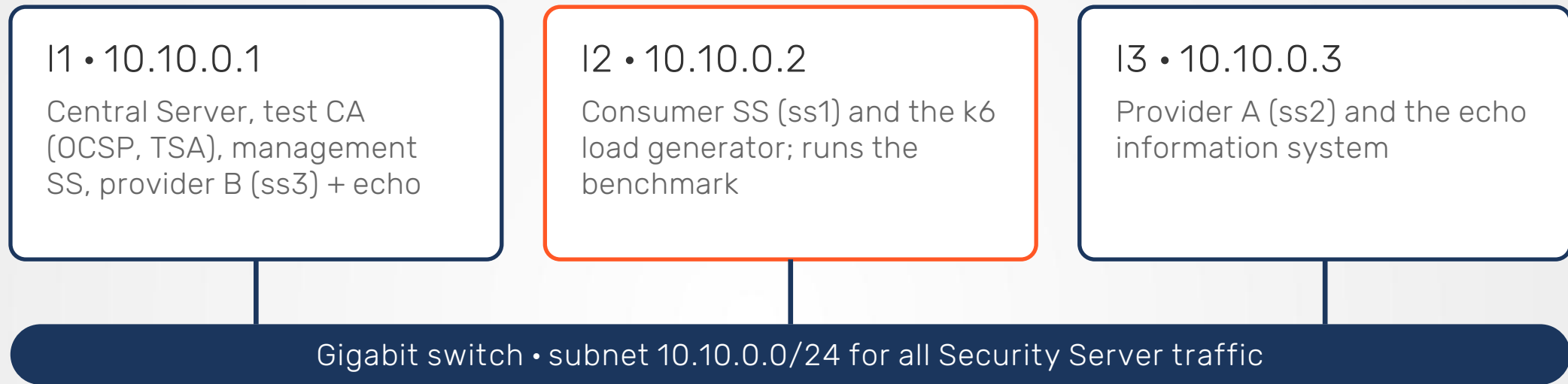
- p50 / p90 / p99 latency, requests per second
- run-to-run spread: IQR and range
- successful requests (HTTP 200)
- bytes on the ss1 ↔ ss2 link only
- TSA and OCSP calls per run
- message-log growth per message
- per-stage times from operational monitoring

Procedure

- 10 KB and 1 MB at 1 client; 10 KB at 4, 16, 32 clients
- burn-in after each configuration change, then warm-up
- 1000 requests per run (200 at 1 MB)
- 5 repetitions per grid, random order
- control run at the start of every repetition
- wait for batch time-stamps, then clean the log
- CPU performance mode, recorded per run



Testbed: three wired machines



- Each container has its own 10.10.0.x address (ipvlan): no NAT, no port publishing, no extra MAC addresses on the university network
- Three Intel i5-8250U laptops; CPU in performance mode; clocks synchronised with chrony
- Remote access over the university VPN; the same stand is also run on one host for comparison

Preliminary results, one client

Config	10 KB p50	Δ vs full	1 MB p50	Δ vs full
direct	0.7 ms	—	10.1 ms	—
full	28.1 ms (IQR 1.5 %)	—	74.1 ms (IQR 4.4 %)	—
nobody	27.4 ms	−0.8 (noise)	39.5 ms	−34.6 (−47 %)
reuse	26.1 ms	−2.0	69.6 ms	−4.5
synctsa	117.0 ms	+88.9	165.8 ms	+91.7

Same grid on one host (same CPU): 29.8 ms at 10 KB, 74.7 ms at 1 MB; per-mechanism differences agree within 1–3 ms.

Medians of 15 runs per point (3 grids $\times$ 5 repetitions). Successful requests: 15 000 of 15 000 at 10 KB, 3 000 of 3 000 at 1 MB. Control run over the night: 27.5–29.0 ms, IQR 1.4 %.

Preliminary. Three machines, 3 $\times$ i5-8250U, CPU performance mode, local test CA and TSA • night of 5–6 October 2026



Connection setup vs per-message work

$\approx 2 \text{ ms}$

mTLS handshake per message (full – reuse, 10 KB, 1 client)

3.3 KB

fewer bytes ss1 $\rightarrow$ ss2 per message with reuse (14.8 $\rightarrow$ 11.5 KB)

$\approx 26 \text{ ms}$

remaining per-message work: signing, verification, logging

By default X-Road opens a new TLS connection for every message. Reusing connections removes the handshake but leaves most of the cost, which is message-level.

Limitations found so far

CPU power mode

Same control run: 79.7 ms in balanced vs 28.7 ms in performance. Earlier runs are not comparable in absolute terms; the mode is now fixed and recorded.

Concurrency

At 4 to 32 clients the spread between runs is large (IQR 25–60 %), so differences between full, nobody and reuse are not resolved there.

Optimistic PKI

Local test CA and TSA on the same network; production intervals and services may differ.

Hardware and scope

Three 2017 laptop CPUs, one local network, no wide-area link. Comparisons are within one environment.

Status and next steps

Done

- Testbed scripted: one host or three machines
- Clean grids on three machines in performance mode
- TLS setup separated from per-message work
- One host vs three machines (RQ3)

In progress

- Same grid on the development laptop
- Per-stage breakdown from operational monitoring

Next

- Concurrency: throughput ceilings only
- Report revision with final tables
- To discuss: plain mTLS baseline? HPC VMs?

Questions and suggestions welcome.